

```
pip install matplotlib
```

```
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.11/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (2.8.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7-
>matplotlib) (1.17.0)
```

```
import random
import matplotlib.pyplot as plt
from itertools import product
```

```
# --- GA Parameters ---
```

```
CHROMOSOME_LENGTH = 10
POPULATION_SIZE = 20
GENERATIONS = 100
TOURNAMENT_SIZE = 3
CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.01
```

```
# Step 1: Fitness Function
```

```
def fitness(chromosome):
    return sum(int(bit) for bit in chromosome)
```

```
# Step 2: Generate Initial Population
```

```
def generate_individual():
    return ''.join(random.choice('01') for _ in
range(CHROMOSOME_LENGTH))
```

```
def generate_population(size):
    return [generate_individual() for _ in range(size)]
```

```

# Step 3: Tournament Selection
def tournament_selection(population):
    selected = []
    for _ in range(POPULATION_SIZE):
        contenders = random.sample(population, TOURNAMENT_SIZE)
        winner = max(contenders, key=fitness)
        selected.append(winner)
    return selected

# Step 4: One-Point Crossover
def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, CHROMOSOME_LENGTH - 1)
        return (parent1[:point] + parent2[point:], parent2[:point] +
parent1[point:])
    return (parent1, parent2)

# Step 5: Bit-Flip Mutation
def mutate(chromosome):
    return ''.join(
        bit if random.random() > MUTATION_RATE else ('0' if bit == '1'
else '1')
        for bit in chromosome
    )

# Step 6: Evolve One Generation
def evolve(population):
    selected = tournament_selection(population)
    next_generation = []
    for i in range(0, POPULATION_SIZE, 2):
        parent1, parent2 = selected[i], selected[i+1]
        offspring1, offspring2 = crossover(parent1, parent2)
        next_generation.extend([mutate(offspring1),
mutate(offspring2)])
    return next_generation

# Step 7: Run Genetic Algorithm
def run_genetic_algorithm():
    population = generate_population(POPULATION_SIZE)
    best_fitness_progress = []

    for generation in range(GENERATIONS):
        population = evolve(population)
        best = max(population, key=fitness)
        best_fitness = fitness(best)
        best_fitness_progress.append(best_fitness)
        if best_fitness == CHROMOSOME_LENGTH:
            print(f" Solution found in generation {generation + 1}:
{best}")
            break

```

```

    return best, best_fitness_progress

# Brute-Force Search
def run_brute_force():
    all_possibilities = [''.join(bits) for bits in product('01',
repeat=CHROMOSOME_LENGTH)]
    best = max(all_possibilities, key=fitness)
    return best, fitness(best)

# Run GA
print("Running Genetic Algorithm...")
ga_solution, fitness_progress = run_genetic_algorithm()

# Run Brute Force
print("\nRunning Brute Force Search...")
bf_solution, bf_fitness = run_brute_force()

# Results
print("\n--- RESULTS ---")
print(f"Genetic Algorithm Solution: {ga_solution}, Fitness: {fitness(ga_solution)}")
print(f"Brute Force Solution: {bf_solution}, Fitness: {bf_fitness}")

# Plot GA progress
plt.plot(fitness_progress)
plt.xlabel("Generation")
plt.ylabel("Best Fitness")
plt.title("GA Progress over Generations")
plt.grid()
plt.show()

```

```

Running Genetic Algorithm...
□ Solution found in generation 3: 1111111111

```

```

Running Brute Force Search...

```

```

--- RESULTS ---
Genetic Algorithm Solution: 1111111111, Fitness: 10
Brute Force Solution:      1111111111, Fitness: 10

```

GA Progress over Generations

